

0.5. La même ligne, avec une boucle for

Vingt lignes presque pareilles deviennent une seule, répétée vingt fois.

1. Ce qu'il faut comprendre

`for (uint8_t i = 0; i < 20; i++)` répète ce qui est entre ses accolades **20 fois**. `i` vaut 0 au premier tour, puis 1, 2... jusqu'à 19.

`i` sert **deux fois** : comme **colonne**, pour avancer de gauche à droite, et comme **indice** dans `ALPHABET`, pour passer de A à T.

`i < 20` : on s'arrête avant la colonne 20, qui n'existe pas, exactement comme au 0.4. Le résultat est le même, en 3 lignes au lieu de 20, et le programme est plus léger.

2. Le programme, morceau par morceau

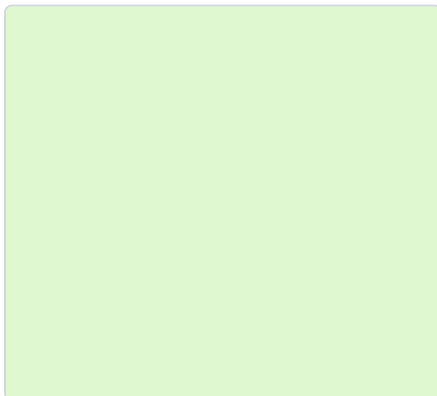
Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1333 octets** de cartouche.

1. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

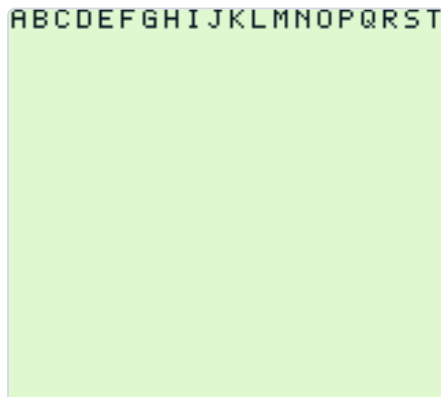
```
int main() {  
    for (uint8_t i = 0; i < 20; i++) {    // 20 tours : les colonnes 0 à 19  
        poser(i, 0, ALPHABET[i]);        // la lettre n° i, dans la colonne i  
    }  
  
    while (true) {  
        image();  
    }  
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

A à T sur toute la première ligne, comme au 0.4.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ la ligne 0 est pleine : A à T
- ✓ rien ne déborde sur la ligne 1

Ensuite : **0.5.1. La même ligne, avec une boucle for — de base, ailleurs** — niveau 0 sur 10. · [gameboy3](#) — écrire une cartouche Game Boy en C++